FraudeLink: Detección de Fraude Financiero mediante Bases de Datos de Grafos

Grupo 8
Joselyn Godínez, Emilio Zamora, Carlos Quesada, Alejandro Castro, Giovanni Calvo Grillo
Escuela de Estadística
Universidad de Costa Rica

***Abstract*— Este trabajo presenta el diseño e implementación de FraudeLink, una solución de detección de fraude financiero basada en bases de datos de grafos. Se utiliza Memgraph con la extensión MAGE para modelar las relaciones entre clientes, cuentas, dispositivos, direcciones IP y comercios, sobre un dataset sintético reproducible de 12,667 nodos y 73,317 relaciones que supera los mínimos exigidos. Se implementaron y validaron siete requisitos técnicos: diseño del esquema lógico del grafo, detección de identidades compartidas por dispositivo o IP, detección de cadenas y ciclos de transferencias, análisis de centralidad (PageRank y grado), detección de comunidades (Louvain), búsqueda de camino mínimo entre cuentas y consulta combinada de riesgo. Se documenta con evidencia empírica que algoritmos como PageRank y Louvain no logran aislar el patrón de fraude sembrado, y se explica por qué desde la naturaleza misma del dataset. Se identificó y resolvió un error de memoria durante la ejecución de consultas de camino variable, y se refinó el criterio de detección de IP compartida hasta obtener un resultado limpio equivalente al de dispositivo compartido.**

Index Terms*— *bases de datos de grafos, detección de fraude, Memgraph, Cypher, MAGE, PageRank, Louvain, lavado de dinero.

I. Introducción

FraudeLink es una fintech que procesa transferencias y pagos digitales. Su equipo de prevención de fraude sospecha que varias cuentas aparentemente independientes comparten dispositivos, direcciones IP y beneficiarios, y que ciertos grupos realizan transferencias circulares para ocultar el origen del dinero [1]. El obstáculo operativo no es la falta de datos, sino la dificultad de explorarlos: las consultas relacionales de múltiples saltos que requeriría esta investigación son costosas de ejecutar y difíciles de mantener [2], una limitación que se retoma con evidencia propia en la Sección XI.

Las consecuencias de no resolverlo son concretas. Un anillo de identidades falsas que comparte un mismo dispositivo puede operar indefinidamente si cada cuenta se evalúa de forma aislada; y una cadena de transferencias circulares, donde ningún monto individual resulta llamativo, permanece invisible para un sistema de alertas que examina transacciones una por una. En ambos casos, la señal de fraude no reside en ninguna entidad individual, sino en la estructura de conexiones entre ellas.

Esta caracterización del problema, formulada íntegramente en términos de conectividad, es la que motiva la elección del modelo de grafos. FraudeLink se implementa sobre Memgraph [3] con la extensión de algoritmos MAGE [4], donde las relaciones son ciudadanos de primera clase. Este trabajo documenta los fundamentos del modelo de grafos, la justificación de la tecnología seleccionada, el diseño del esquema, los siete requisitos implementados, la metodología de pruebas, una comparación explícita contra el enfoque relacional, y una discusión crítica y honesta sobre qué técnicas logran aislar los patrones de fraude sembrados y cuáles no.

II. Fundamentos del Modelo de Grafos

A. Modelo de Datos

Una base de datos de grafos organiza la información en nodos (entidades) y aristas o relaciones (conexiones entre entidades), donde ambos pueden llevar propiedades en forma de pares clave-valor. Este modelo se conoce como grafo de propiedades (property graph). A diferencia del modelo relacional, donde las conexiones entre tablas se resuelven en tiempo de consulta mediante operaciones de unión sobre claves foráneas, en un grafo la relación es un objeto almacenado explícitamente, con su propia identidad, tipo, dirección y atributos.

Esta diferencia tiene consecuencias prácticas directas. En un modelo relacional, recorrer tres saltos de relación implica tres uniones sucesivas cuyo costo crece con el tamaño de las tablas involucradas. En un grafo, cada nodo mantiene referencias directas a sus vecinos, de modo que el recorrido es una operación local: el costo depende del número de vecinos visitados, no del tamaño total de la base. Esta propiedad se conoce como adyacencia sin índice (index-free adjacency) [2], y es la razón central por la que los grafos resultan adecuados para problemas donde la pregunta de interés es sobre conexiones y no sobre agregaciones de filas.

B. Mecanismo de Consulta

El lenguaje de consulta predominante en bases de datos de grafos de propiedades es Cypher, un lenguaje declarativo cuya sintaxis representa visualmente los patrones del grafo. La expresión (a)-[:REL]->(b) describe literalmente un nodo a conectado a un nodo b mediante una relación dirigida de tipo REL. Esta correspondencia entre la forma del código y la forma del dato hace que consultas complejas de múltiples saltos se expresen de manera considerablemente más legible

que su equivalente en SQL con expresiones de tabla comunes recursivas.

Cypher incorpora además construcciones específicas para grafos que no tienen equivalente directo en SQL: caminos de longitud variable (*n..m), que permiten recorrer entre n y m saltos sin conocer la profundidad de antemano; funciones de camino como nodes() y relationships(), que extraen los elementos recorridos; y algoritmos de recorrido como búsqueda en anchura (BFS) invocables directamente desde el patrón. Estas construcciones son las que habilitan, en este trabajo, la detección de ciclos y el cálculo de caminos mínimos.

C. Consistencia y Escalabilidad

Memgraph ofrece garantías ACID en su modo de almacenamiento por defecto, IN_MEMORY_TRANSACTIONAL. Mediante control de concurrencia multiversión (MVCC), cada escritura genera un objeto Delta; estos Deltas son la base sobre la que se construyen los write-ahead logs (durabilidad) y el nivel de aislamiento por defecto, Snapshot Isolation, que garantiza que toda lectura observa una vista consistente de la base [5]. Memgraph ofrece además un modo alternativo, IN_MEMORY_ANALYTICAL, optimizado para ingestión masiva, que renuncia explícitamente a estas garantías; la implementación de este trabajo se ejecuta sobre el modo transaccional por defecto.

En cuanto a escalabilidad, la documentación oficial de Memgraph establece explícitamente que el motor no soporta particionamiento (sharding) horizontal de escritura, y justifica esta decisión de diseño señalando que las consultas de múltiples saltos, características de las cargas de trabajo de grafos, requerirían atravesar particiones (cross-shard traversal) para resolverse, lo que degradaría significativamente el rendimiento y las garantías de consistencia [6]. En su lugar, Memgraph escala lecturas horizontalmente mediante réplicas de solo lectura, y escala escrituras verticalmente (más núcleos y memoria en la instancia principal). Esta característica es relevante para el caso presente: como se documenta en la Sección VI, las consultas de camino variable sobre un volumen de decenas de miles de relaciones ya evidencian límites de memoria en una instancia individual, consistente con un diseño que privilegia el escalamiento vertical sobre el particionamiento.

D. Ventajas, Limitaciones y Casos de Uso

Entre las ventajas del modelo se encuentran el rendimiento estable en consultas de múltiples saltos con independencia del tamaño total de la base, la flexibilidad de esquema, que permite agregar nuevos tipos de nodo o relación sin migraciones estructurales, y la expresividad del lenguaje de consulta para patrones de conexión. Entre las limitaciones: el rendimiento inferior frente a motores relacionales u orientados a columnas en agregaciones masivas sobre grandes volúmenes homogéneos, la dificultad de particionamiento horizontal ya mencionada, y el costo de las consultas de camino de longitud variable sin acotar, que pueden generar una explosión combinatoria, como se evidenció empíricamente en este trabajo.

Los casos de uso característicos incluyen detección de fraude, motores de recomendación, análisis de redes sociales, gestión de identidad y accesos, trazabilidad de cadenas de suministro y grafos de conocimiento. El caso FraudeLink pertenece a la primera categoría, y su formulación original (cuentas que comparten dispositivos, transferencias circulares, caminos entre cuentas sospechosas) está expresada directamente en términos de conectividad, lo que justifica la elección del modelo.

III. Tecnología Seleccionada

Se seleccionó Memgraph [3] como motor de base de datos, con la biblioteca de algoritmos MAGE [4]. La elección se justifica en los siguientes criterios, todos verificables en la implementación entregada.

La solución se compone de tres piezas. Un script en Python genera el conjunto de datos con la biblioteca Faker [7] y lo carga en la base mediante el protocolo Bolt (puerto 7687), usando el driver neo4j. Memgraph con MAGE se ejecuta como una única instancia dentro de un contenedor Docker, en el modo IN_MEMORY_TRANSACTIONAL descrito en la Sección II-C. Las consultas en Cypher se escriben, ejecutan y visualizan desde Memgraph Lab (puerto 3000), que es también la herramienta con la que se midieron los tiempos de la Sección XII-B y se obtuvo la Fig. 1.

A. Correspondencia con el Modelo Asignado

Memgraph es una base de datos de grafos de propiedades nativa: no es una plataforma multimodelo que ofrezca grafos como una capa sobre otro modelo de almacenamiento. Toda la solución implementada opera sobre nodos y relaciones, y la totalidad de las consultas están escritas en Cypher, sin recurrir a ningún otro paradigma de consulta.

B. Disponibilidad de Algoritmos

MAGE provee una biblioteca de algoritmos de grafos invocables como procedimientos desde Cypher. Se verificó empíricamente su disponibilidad en la instancia mediante CALL mg.procedures(), que reportó 309 procedimientos instalados, incluyendo pagerank.get y community_detection.get, ambos utilizados en este trabajo (Secciones VII y VIII).

C. Reproducibilidad del Entorno

El despliegue se realiza mediante contenedores Docker con la imagen memgraph-mage, que integra Memgraph 3.12.0 con la biblioteca de algoritmos MAGE, lo que garantiza que cualquier integrante del equipo o evaluador obtenga un entorno idéntico siguiendo únicamente las instrucciones del repositorio, sin credenciales privadas ni licencias de pago. Un segundo contenedor levanta Memgraph Lab y se enlaza con el primero, de modo que un único comando pone en marcha la base y su interfaz de consulta. La conexión desde Python se realiza mediante el protocolo Bolt usando el driver neo4j, compatible con Memgraph; se optó por este driver en lugar de gqlalchemy porque este

último requiere compilar componentes nativos con dependencias de C++ y OpenSSL que no estaban disponibles de manera uniforme en todos los equipos del grupo.

Las mediciones de desempeño reportadas en la Sección XII-B se obtuvieron sobre un equipo portátil con procesador Intel Core i9 de ocho núcleos a 2.4 GHz y 32 GB de memoria DDR4 a 2667 MHz, bajo macOS con Docker Desktop. Conviene precisar que en esa configuración Memgraph no dispone de la totalidad de la memoria del equipo: el contenedor se ejecuta dentro de una máquina virtual a la que Docker asigna únicamente una fracción de la RAM del sistema, y el motor fija su propio límite en torno al 90% de lo que esa máquina virtual le expone. De ahí proviene el tope de 7.75 GiB que aparece en el error documentado en la Sección VI-B, muy inferior a los 32 GB instalados en el equipo.

D. Diferenciación frente al Equipo Rival

El caso establece que los dos equipos asignados al mismo problema deben implementar con tecnologías distintas. El equipo rival trabaja con Neo4j; la selección de Memgraph cumple esta condición mientras mantiene la correspondencia con el modelo de grafos asignado.

IV. Modelo del Grafo y Conjunto de Datos

El esquema propuesto consta de cinco tipos de nodo (Cliente, Cuenta, Dispositivo, IP, Comercio) y cinco relaciones dirigidas, cada una con un sentido semántico claro que habilita las consultas de camino requeridas por los requisitos posteriores:

- (Cliente)-[:POSEE]->(Cuenta)
- (Cuenta)-[:USA_DISPOSITIVO]->(Dispositivo)
- (Dispositivo)-[:CONECTA_DESDE]->(IP)
- (Cuenta)-[:TRANSFIERE_A {monto, fecha}]->(Cuenta)
- (Cuenta)-[:PAGA_EN {monto, fecha}]->(Comercio)

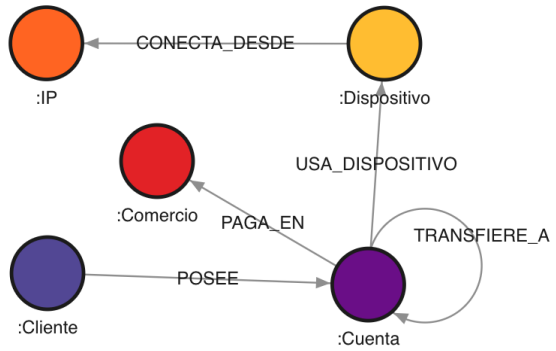


Fig. 1. Esquema lógico del grafo, visualizado en Memgraph Lab.

La Fig. 1 muestra el esquema lógico tal como lo reporta Memgraph sobre la base cargada, lo que confirma que la estructura implementada corresponde al diseño propuesto. La direccionalidad no es cosmética: es lo que permite las consultas de rango variable (*n..m) sobre TRANSFIERE_A, la detección de ciclos volviendo al origen y el camino mínimo entre cuentas sospechosas. Nótese que TRANSFIERE_A es una relación reflexiva a nivel de tipo, ya que conecta Cuenta con Cuenta, lo que habilita precisamente los recorridos cíclicos analizados en la Sección VI.

En cuanto a las propiedades, cada nodo almacena solo lo necesario para identificarlo: un identificador (id, o direccion en el caso de IP) y, en Cliente y Comercio, un nombre. El monto y la fecha se almacenan en las relaciones TRANSFIERE_A y PAGA_EN, y no en un nodo intermedio de tipo Transferencia, porque cada transferencia vincula exactamente dos cuentas y las consultas de ciclos y camino mínimo recorren directamente de cuenta a cuenta; con un nodo intermedio, un ciclo de cinco transferencias pasaría a ser un camino de diez saltos. La contrapartida es que una transferencia no puede vincularse a otra entidad, por ejemplo al dispositivo desde el que se originó. La frecuencia no se almacena porque se obtiene contando relaciones en tiempo de consulta, como hace la medida de grado de la Sección VII.

La generación del conjunto de datos es reproducible: se emplea Python con la biblioteca Faker [7] y una semilla fija (SEED=42), de modo que cualquier integrante del equipo o evaluador obtiene exactamente los mismos datos. La composición resultante se detalla en la Tabla I.

TABLA I
Composición del Conjunto de Datos Generado

Tipo de nodo	Cantidad
Cliente	3,000
Cuenta	6,065
Dispositivo	1,801
IP	1,401
Comercio	400
Total de nodos	12,667
Tipo de relación	**Cantidad**
POSEE	6,065
USA_DISPOSITIVO	6,115
CONECTA_DESDE	6,056
TRANSFIERE_A	40,081
PAGA_EN	15,000
Total de relaciones	73,317

Ambos totales superan los mínimos exigidos por el caso (10,000 nodos y 50,000 relaciones). El generador siembra además dos patrones de fraude deliberados que sirven como verdad de referencia para validar las consultas: un dispositivo (DISP_FRAUDE_00001) asociado a una IP fija (190.10.99.99), ambos compartidos por 50 cuentas de clientes distintos; y 20 ciclos de transferencias de 3 a 5 saltos con montos entre $1,000 y $9,000, frente a un rango de $5 a $5,000 en las transferencias normales.

V. Detección de Identidades Compartidas

En el grafo, el dispositivo está a un salto de distancia de la Cuenta y la IP a dos (Cuenta→Dispositivo→IP), por lo que el requisito se resuelve con dos consultas separadas, una para cada medio.

A. Dispositivos Compartidos

Se calibró un umbral empírico de 14 clientes distintos por dispositivo:

```
MATCH (cli:Cliente)-[:POSEE]->(cta:Cuenta)
      -[:USA_DISPOSITIVO]->(d:Dispositivo)
WITH d, collect(DISTINCT cli.id) AS clientes,
     collect(DISTINCT cta.id) AS cuentas
WHERE size(clientes) > 14
RETURN d.id AS dispositivo,
       size(clientes) AS num_clientes_distintos,
       clientes, cuentas
ORDER BY num_clientes_distintos DESC;
```

La calibración del umbral se realizó observando el comportamiento del resultado ante distintos valores de corte, resumidos en la Tabla II.

TABLA II
CALIBRACIÓN DEL UMBRAL DE DISPOSITIVOS COMPARTIDOS

Umbral	Resultados	Primer resultado
> 1	1,542	DISP_FRAUDE_00001 (50)
> 8	11	DISP_FRAUDE_00001 (50)
> 9.66 (3σ)	7	DISP_FRAUDE_00001 (50)
> 14	1	DISP_FRAUDE_00001 (50)

Sin filtro se obtienen 1,542 dispositivos compartidos por al menos dos clientes, resultado del reparto aleatorio de 6,065 cuentas entre 1,800 dispositivos. La calibración a >14 aísla exactamente el patrón sembrado.

También se evaluó el criterio estadístico de tres desviaciones estándar sobre la media, que reemplaza el corte fijo por uno que se ajusta automáticamente a cada dataset. La media de clientes distintos por dispositivo es 3.52 y la desviación estándar 2.05, lo que sitúa el umbral en 9.66. Como muestra la Tabla II, este criterio devuelve 7 dispositivos: el sembrado y seis dispositivos normales con 10 a 12 clientes, producto del reparto aleatorio.

La discrepancia entre ambos criterios tiene una explicación estadística. Como el generador asigna cada cuenta a un dispositivo de forma uniforme, el número de cuentas por dispositivo sigue aproximadamente una distribución de Poisson de parámetro $\lambda = 6{,}065/1{,}800 = 3.37$, modelo consistente con los datos: predice una media de 3.49 entre los dispositivos que reciben al menos una cuenta, frente a los 3.52 observados. Bajo ese modelo, la probabilidad de que un dispositivo normal supere los 14 clientes es de unas 3 en un millón, equivalente a 0.005 dispositivos esperados entre los 1,800, de modo que el umbral de 14 queda justificado a priori y no solo por inspección del resultado. El mismo modelo explica el criterio 3σ: con un corte en 9.66 se esperan 4.6 dispositivos normales por encima del umbral, cifra compatible con los 6 observados, porque la distribución de Poisson es asimétrica y de cola más pesada que la normal en este rango.

En este dataset, por tanto, el criterio estadístico resulta menos preciso que el corte calibrado empíricamente, y requeriría una verificación adicional para separar el dispositivo sembrado del ruido.

B. IP Compartida

Filtrar por size(clientes) > 14, el mismo umbral usado para dispositivos, no es suficiente para aislar la IP sembrada: devuelve 885 resultados, y aunque la IP fraudulenta (190.10.99.99) aparece entre ellos con 50 clientes, queda en la posición 7, sin distinguirse del resto. La causa está en CONECTA_DESDE: conecta cada Dispositivo a una IP elegida al azar por cada cuenta que lo usa, no a una IP fija por dispositivo, de modo que el camino Cuenta→Dispositivo→IP hereda el fan-out completo de clientes del dispositivo hacia todas sus IPs.

Agrupar solo por IP y exigir, además de muchos clientes en total, que el promedio de clientes por dispositivo conectado a esa IP también sea alto, sí aísla el caso sembrado:

```
MATCH (cli:Cliente)-[:POSEE]->(cta:Cuenta)
      -[:USA_DISPOSITIVO]->(disp:Dispositivo)
      -[:CONECTA_DESDE]->(ip:IP)
WITH ip,
     count(DISTINCT cli) AS num_clientes,
     collect(DISTINCT cta.id) AS cuentas,
     count(DISTINCT disp) AS num_dispositivos
WITH ip, num_clientes, cuentas, num_dispositivos,
     toFloat(num_clientes)/num_dispositivos AS prom
WHERE num_clientes > 12 AND prom > 12
RETURN ip.direccion, num_clientes,
       num_dispositivos, prom
ORDER BY num_clientes DESC;
```

TABLA III
COMPARACIÓN DE CRITERIOS PARA IP COMPARTIDA

Criterio de filtrado	Resultados
size(clientes) > 14	885
num_clientes > 12 y prom > 12	1

Con la segunda condición, el resultado es único: 190.10.99.99, con 50 clientes, 1 dispositivo, y promedio de 50 clientes por dispositivo, el mismo nivel de aislamiento que el dispositivo compartido en la Parte A.

C. Verificación Cruzada

Se comparó si las 50 cuentas del dispositivo fraudulento coinciden con las 50 cuentas de la IP fraudulenta. Resultado: 50/50/50, coincidencia total. Esto indica que, para el caso sembrado, dispositivo e IP representan la misma señal unida; el ruido observado con el filtro simple de la Parte B provenía del resto del grafo, no del caso de fraude.

VI. CADENAS Y CICLOS DE TRANSFERENCIAS

El requisito pide una cadena o un ciclo de al menos tres saltos. El patrón es relevante porque un ciclo permite mover dinero a través de varias cuentas hasta devolverlo a la de origen, de modo que se pierde el rastro de su procedencia aunque ningún monto individual resulte llamativo.

A. Ciclo (Vuelve al Origen)

```
MATCH camino = (a:Cuenta)-[:TRANSFIERE_A*3..5]->(a)
RETURN a.id, length(camino) AS num_saltos,
       [n IN nodes(camino) | n.id] AS ruta,
       [r IN relationships(camino) | r.monto]
LIMIT 10;
```

Sin ORDER BY, la consulta respondió en 18 ms (Tabla IX): el LIMIT 10 permite a Memgraph detenerse apenas encuentra los primeros diez ciclos, que en esa ejecución fueron 8 de 5 saltos y 2 de 4, en el orden en que los halla su recorrido interno y no por longitud. Al ordenar por longitud (ORDER BY num_saltos DESC), la misma consulta tardó 15.71 segundos sobre ~40,000 transferencias, porque ordenar obliga a enumerar antes todos los ciclos; es del mismo orden de magnitud que las demás consultas que recorren el conjunto completo (8.92 s y 9.14 s en la Tabla IX), aunque algo más lenta por el costo adicional de ordenar explícitamente las 14,779 filas resultantes.

Sin límite, la consulta reporta 14,779 filas. Esta cifra sobreestima el número de ciclos distintos: el patrón (a)-[*3..5]->(a) permite que a sea cualquier nodo del ciclo, de modo que un ciclo de k cuentas aparece k veces, una por cada punto de partida posible.

Aun con esa salvedad, el orden de magnitud muestra que los ciclos de transferencias ocurren por azar con enorme frecuencia en un grafo de este tamaño y densidad. El número observado coincide, de hecho, con el que predice un modelo aleatorio: siendo 6.6 el grado de salida promedio, el número esperado de caminos cerrados de longitud k es del orden de 6.6 elevado a k, lo que para k entre 3 y 5 suma unas 14,800 filas, frente a las 14,779 observadas. Los 20 ciclos sembrados están incluidos entre esas filas, pero su forma por sí sola no los distingue del ruido, igual que ocurrió con la IP compartida en la Sección V-B.

Para aislarlos se utiliza el monto. Las transferencias normales van de $5 a $5,000, y las sembradas de $1,000 a $9,000; exigir que todas las transferencias del ciclo superen $5,000 garantiza que el ciclo proviene del generador de fraude:

```
MATCH camino = (a:Cuenta)-[:TRANSFIERE_A*3..5]->(a)
WHERE ALL(r IN relationships(camino)
          WHERE r.monto > 5000)
RETURN a.id, length(camino) AS num_saltos,
       [n IN nodes(camino) | n.id] AS ruta;
```

TABLA IV
EMBUDO DE DETECCIÓN DE CICLOS

Etapa	Resultado
Filas con el patrón (3 a 5 saltos)	14,779
Filas con todos los montos > $5,000	7
Ciclos distintos tras agrupar	2
Esperado entre los 20 sembrados	1 a 2

La Tabla IV resume el proceso. Las 7 filas corresponden a solo 2 ciclos distintos, uno de 3 saltos y otro de 4, por el mismo efecto de sobreconteo; se verificó agrupando las filas según su conjunto de cuentas. La cifra coincide con lo esperado: la probabilidad de que un ciclo sembrado de k saltos tenga todos sus montos por encima de $5,000 es 0.5^k, lo que sobre 20 ciclos de 3 a 5 saltos da en promedio entre 1 y 2 ciclos.

Estos 2 ciclos pertenecen con certeza al patrón sembrado, ya que ninguna transferencia normal supera $5,000, pero representan un piso mínimo: los otros 18 tienen montos mixtos y este método no logra separarlos del ruido.

B. Cadena Lineal

La cadena lineal (sin volver al origen) requiere una consulta análoga con a<>b. A diferencia del ciclo de la Parte A, aquí no hay restricción de retorno al origen, por lo que el espacio de búsqueda es mayor. Ejecutarla con ORDER BY produce Memory limit exceeded: 8.40GiB > 7.75GiB permitido, límite cuyo origen se detalla en la Sección III-C, porque Memgraph debe materializar todas las cadenas de 3 a 4 saltos antes de poder ordenarlas. Omitir el ORDER BY evita este límite, a costa de que los 10 resultados devueltos no sean necesariamente los más largos ni los más relevantes, sino los primeros que el motor encuentra en su recorrido interno.

VII. ANÁLISIS DE CENTRALIDAD

Se implementaron dos medidas complementarias: PageRank [8] (recursivo, provisto por MAGE [4]) y grado simple (conteo directo).

```
CALL pagerank.get() YIELD node, rank
WHERE node:Cuenta
RETURN node.id, rank ORDER BY rank DESC LIMIT 10;
```

Los valores de PageRank del top 10 oscilaron entre 0.000164 y 0.000205: la primera cuenta supera a la décima en apenas un 25%, un margen demasiado estrecho para separar un grupo destacado del resto.

```
MATCH (cta:Cuenta)-[:USA_DISPOSITIVO]->
      (:Dispositivo {id: "DISP_FRAUDE_00001"})
WITH collect(cta.id) AS sospechosas
CALL pagerank.get() YIELD node, rank
WHERE node:Cuenta
WITH sospechosas, node.id AS cuenta, rank
ORDER BY rank DESC LIMIT 10
RETURN cuenta, rank,
       cuenta IN sospechosas AS es_sospechosa;
```

La verificación cruzada de PageRank mostró que 1 de las 10 cuentas del top (CLI_02858_CTA_2, en la posición 5) sí pertenece al grupo de 50 sospechosas del dispositivo fraudulento. Sobre una probabilidad base de que una cuenta cualquiera del dataset sea sospechosa (50/6,065 ≈ 0.82%), el valor esperado bajo azar puro para una muestra de 10 es de 0.08 coincidencias, y la probabilidad de observar al menos una es cercana al 8%, por encima del umbral convencional del 5% con el que se descartaría el azar. La coincidencia observada es, por tanto, compatible con el azar y no constituye evidencia de que PageRank aísle el patrón.

El grado simple, calculado con OPTIONAL MATCH + count() debido a que Memgraph no permite size((c)-[:TIPO]-()) fuera de un WHERE, devolvió un top 10 con 26 a 28 transferencias contra un promedio esperado de 13.2, aproximadamente el doble. Ese valor tampoco resulta anómalo: bajo el mismo modelo de Poisson aplicado en la Sección V-A, ahora con media 13.2, se esperan 7.4 cuentas con 26 o más transferencias entre las 6,065 del grafo, de modo que un top 10 situado en ese rango es exactamente lo que produce el azar. La Tabla V compara ambas medidas.

TABLA V
COMPARACIÓN DE MEDIDAS DE CENTRALIDAD

Medida	Rango top 10	Razón 1.º / 10.º	Coincidencias/10
PageRank	0.000164–0.000205	1.25	1
Grado simple	26–28	1.08	0

La verificación cruzada del grado reveló que ninguna de las 10 cuentas con mayor grado pertenece al grupo de 50 sospechosas; bajo el mismo cálculo de probabilidad aplicado a PageRank, el resultado más probable por puro azar para esta medida es también 0 coincidencias sobre 10.

La diferencia entre 1 coincidencia en PageRank y 0 en grado simple no debe leerse como que PageRank funcione mejor: ambas medidas tienen el mismo valor esperado bajo azar, 0.08 sobre una muestra de 10. Las dos coinciden en la misma conclusión: la señal del fraude sembrado no reside en la actividad transaccional, sino en la identidad compartida (Sección V).

VIII. DETECCIÓN DE COMUNIDADES

Se aplicó el algoritmo Louvain [9] de MAGE [4] (community_detection.get) sobre el grafo completo, filtrando después los nodos de tipo Cuenta. El algoritmo agrupa nodos que están más conectados entre sí que con el resto del grafo, optimizando una métrica conocida como modularidad. La Tabla VI presenta las diez comunidades de mayor tamaño.

TABLA VI
DIEZ COMUNIDADES DE MAYOR TAMAÑO

ID comunidad	Cuentas	ID comunidad	Cuentas
7	460	11	276
8	403	19	275
9	338	14	272
21	313	6	267
1	313	3	257

Las diez comunidades mayores agrupan 3,174 cuentas, aproximadamente el 52% de las 6,065 existentes. Este resultado debe leerse con cautela: Louvain siempre devuelve una partición, incluso sobre un grafo aleatorio, y como el generador crea las transferencias entre cuentas elegidas al azar, estas comunidades reflejan sobre todo fluctuaciones del muestreo y no grupos con significado de negocio.

La verificación cruzada de las 50 cuentas sospechosas contra sus comunidades asignadas produjo el hallazgo más ilustrativo del trabajo. La distribución resultante se resume en la Tabla VII.

TABLA VII
DISTRIBUCIÓN DE LAS 50 CUENTAS SOSPECHOSAS POR COMUNIDAD

Cuentas en una misma comunidad	7	4	3	2	1
Núm. de comunidades	1	3	5	6	4

Las 50 cuentas se dispersaron en 19 comunidades distintas, con la más concentrada (la comunidad 18) conteniendo apenas 7 de ellas, y cuatro comunidades conteniendo una sola cuenta sospechosa. Aunque Louvain considera todas las relaciones del grafo, la mayoría de las conexiones de cada cuenta son transferencias (unas 13 en promedio, frente a una o dos con dispositivos), de modo que el único enlace compartido con DISP_FRAUDE_00001 no alcanza para agruparlas. Son señales estructurales independientes en este dataset: cada requisito capta un ángulo distinto del fraude, y no debe esperarse que todas las técnicas converjan sobre las mismas cuentas.

Un detalle adicional refuerza esta conclusión: la comunidad 18, que concentra la mayor cantidad de cuentas sospechosas, no figura entre las diez comunidades más grandes de la Tabla VI. Es decir, ni siquiera la agrupación con mayor presencia de fraude resulta estructuralmente prominente en el grafo de transferencias. Cabe señalar, no obstante, que tres de las diez cuentas de mayor riesgo combinado identificadas en la Sección X pertenecen a esa misma comunidad 18; dado que la comunidad contiene 7 de las 50 cuentas sospechosas, esta coincidencia no es estadísticamente concluyente, pero sugiere una línea de análisis para trabajo futuro con volúmenes mayores.

IX. CAMINO MÍNIMO ENTRE CUENTAS

Se buscó el camino más corto entre dos cuentas sospechosas, ambas identificadas en la Sección X por su alto riesgo combinado, usando la expansión *BFS de largo variable, que es el mecanismo nativo de Memgraph para caminos mínimos:

```
MATCH (c1:Cuenta {id: "CLI_02908_CTA_0"}),
      (c2:Cuenta {id: "CLI_00717_CTA_0"})
MATCH camino = (c1)-[:TRANSFIERE_A*BFS..10]->(c2)
RETURN [n IN nodes(camino) | n.id] AS ruta,
       [r IN relationships(camino) | r.monto],
       length(camino) AS saltos;
```

Resultado: camino de 5 saltos con 4 cuentas intermediarias (CLI_00632_CTA_0 → CLI_02231_CTA_0 → CLI_01586_CTA_0 → CLI_02828_CTA_1), con montos $701.34, $2,301.42, $3,098.85, $4,444.45 y $4,241.30. Ambas cuentas de origen y destino son, por sí solas, dos de las cuentas de mayor riesgo combinado del grupo sospechoso (Sección X); el grafo revela que además están conectadas entre sí a través de una cadena de intermediarios, un vínculo que un análisis relacional tradicional demandaría múltiples joins recursivos para descubrir.

Conviene, no obstante, matizar el hallazgo. Con un grado de salida promedio de 6.6 transferencias por cuenta, la distancia típica entre dos cuentas cualesquiera del grafo es del orden de $\ln(6{,}065)/\ln(6.6) \approx 4.6$ saltos, de modo que la existencia de un camino de 5 saltos no es por sí sola evidencia de vínculo. El valor operativo de la consulta está en que devuelve, en milisegundos, la lista concreta de intermediarios sobre la que continuar la investigación.

X. Consulta de Riesgo Combinada

Combinando estructura (pertenencia al grupo del dispositivo sospechoso) y propiedades (monto total transferido), se construyó un ranking de riesgo:

```
MATCH (cta:Cuenta)-[:USA_DISPOSITIVO]->
      (d:Dispositivo {id: "DISP_FRAUDE_00001"})
MATCH (cta)-[t:TRANSFIERE_A]->(:Cuenta)
WITH cta.id AS cuenta,
     count(t) AS num_transferencias,
     round(sum(t.monto)) AS monto_total
RETURN cuenta, monto_total, num_transferencias,
       round(monto_total/num_transferencias) AS prom
ORDER BY monto_total DESC LIMIT 10;
```

TABLA VIII
Ranking de Riesgo Combinado (Top 10)

Cuenta	Monto total	Transf.	Promedio
CLI_02908_CTA_0	$31,822	10	$3,182
CLI_00717_CTA_0	$30,801	11	$2,800
CLI_02394_CTA_0	$27,339	9	$3,038
CLI_02657_CTA_1	$26,668	9	$2,963
CLI_00620_CTA_2	$26,368	12	$2,197
CLI_02926_CTA_1	$25,425	9	$2,825
CLI_02615_CTA_0	$25,030	8	$3,129
CLI_02460_CTA_0	$23,344	8	$2,918
CLI_00540_CTA_1	$22,394	9	$2,488
CLI_02667_CTA_2	$22,270	9	$2,474

La cuenta de mayor riesgo combinado fue CLI_02908_CTA_0, con $31,822 en 10 transferencias (promedio $3,182). Los promedios de todo el top 10 ($2,197 a $3,182) se ubican dentro del rango normal de transferencias ($5 a $5,000), y alrededor de su valor esperado: como el generador asigna las transferencias normales con montos uniformes en ese rango, el promedio esperado de cualquier cuenta es de aproximadamente $2,500, independientemente de si pertenece o no al grupo sospechoso.

Esta observación obliga a interpretar el resultado con cautela. El generador no asignó transferencias especiales a las 50 cuentas del dispositivo fraudulento, de modo que su ranking por monto total refleja la variación aleatoria de montos y cantidades, y no un patrón de acumulación oculto. El valor de la consulta está en su estructura: combina una señal estructural (pertenencia a una identidad compartida, Sección V) con una propiedad de las relaciones (monto transferido) para priorizar, dentro de un grupo ya sospechoso, qué cuentas investigar primero. En un sistema real, donde las cuentas fraudulentas sí podrían exhibir patrones de monto distintos, ordenaría la investigación por exposición económica. Parte, además, del identificador del dispositivo ya conocido, por lo que depende de la detección previa de la Sección V y no constituye una detección independiente.

XI. Comparación con el Enfoque Relacional

El caso requiere discutir cómo cambiarían las consultas y sus costos si el problema se resolviera con joins relacionales tradicionales [1]. La comparación que sigue es analítica: no se implementó una versión relacional paralela, de modo que las estimaciones de costo se derivan de la estructura de las operaciones y no de mediciones empíricas.

A. Traducción de las Consultas

Supóngase un esquema relacional equivalente con las tablas CLIENTE, CUENTA, DISPOSITIVO, IP y COMERCIO para las entidades, y TRANSFERENCIA(cuenta_origen, cuenta_destino, monto, fecha), PAGO(cuenta_id, comercio_id, monto, fecha), USA_DISPOSITIVO(cuenta_id, dispositivo_id) y CONECTA_DESDE(dispositivo_id, ip_id) para las relaciones. La consulta de identidad compartida (Sección V-A) se traduce con relativa naturalidad, ya que involucra únicamente tres uniones y una agregación:

```
SELECT d.id, COUNT(DISTINCT c.id) AS n_clientes
FROM CLIENTE c
  JOIN CUENTA ct ON ct.cliente_id = c.id
  JOIN USA_DISPOSITIVO u ON u.cuenta_id = ct.id
  JOIN DISPOSITIVO d ON d.id = u.dispositivo_id
GROUP BY d.id HAVING COUNT(DISTINCT c.id) > 14;
```

La diferencia aparece en las consultas de camino. El ciclo de 3 a 5 saltos (Sección VI) no admite una traducción directa: requiere una expresión de tabla común recursiva (WITH RECURSIVE) que acumule la ruta recorrida, controle la profundidad, detecte el retorno al origen y evite ciclos infinitos manteniendo una lista de nodos visitados. La consulta equivalente en Cypher, MATCH camino = (a:Cuenta)-[:TRANSFIERE_A*3..5]->(a), expresa el mismo predicado en una línea, y la funcionalidad de control de profundidad y de recolección de la ruta es provista por el motor.

B. Implicaciones de Costo

El número de caminos que deben explorarse en un recorrido de k saltos crece de forma multiplicativa con la profundidad, y esto ocurre por igual en ambos modelos, ya que depende de la estructura de los datos y no del motor. Como el recorrido sigue relaciones dirigidas, el factor relevante es el grado de salida promedio: con aproximadamente 40,000 transferencias sobre 6,065 cuentas, cada cuenta emite en promedio 6.6 transferencias, de modo que un recorrido de 4 saltos desde una sola cuenta explora del orden de 6.6 elevado a 4, es decir, unos 1,900 caminos, y varios órdenes de magnitud más si el recorrido no se ancla en un punto de partida fijo.

La diferencia entre modelos reside en el costo de cada salto. En el modelo relacional, avanzar un salto implica una unión sobre la tabla de transferencias: aun con índices sobre las columnas de unión, cada paso requiere una búsqueda cuyo costo crece con el tamaño de la tabla, y los resultados intermedios deben materializarse antes de aplicar los filtros finales. En el modelo de grafos, la adyacencia sin índice descrita en la Sección II [2] convierte ese salto en seguir una referencia directa almacenada en el nodo, con un costo independiente del tamaño total de la tabla de relaciones. El número de caminos explorados es el mismo, pero cada uno se recorre más barato, y eso es lo que hace viable expresar directamente consultas como la del camino mínimo (Sección

IX), resuelta con una búsqueda en anchura acotada, frente a la alternativa relacional de una recursión con condición de parada explícita.

Conviene, no obstante, matizar esta comparación con la evidencia propia de este trabajo. El error de memoria documentado en la Sección VI-B demuestra que el modelo de grafos no es inmune a la explosión combinatoria: cuando la consulta obliga a materializar todos los caminos posibles, como ocurre al ordenar resultados de longitud variable, el costo se vuelve prohibitivo también en Memgraph. La ventaja del modelo de grafos no reside en eliminar el problema combinatorio, sino en expresar el recorrido de forma directa y en resolver eficientemente los casos acotados, que son la mayoría de las consultas operativas de detección de fraude.

C. Mantenibilidad

Más allá del costo de ejecución, el caso señala que las consultas relacionales de múltiples saltos resultan difíciles de mantener [1]. La evidencia de este trabajo lo respalda desde el lado opuesto: agregar un salto al análisis de identidad compartida, pasando de dispositivo (un salto) a IP (dos saltos), requirió extender el patrón de Cypher con un único tramo adicional. La modificación equivalente en SQL habría implicado añadir dos uniones y revisar las cláusulas de agrupación, un cambio de mayor superficie sobre el que es más fácil introducir errores.

XII. Metodología de Pruebas y Evidencias

A. Estrategia de Validación

La validación de las consultas no se basó en inspección visual de resultados plausibles, sino en contrastarlos contra una verdad de referencia conocida. Como el generador de datos siembra patrones de fraude deliberados con identificadores fijos (DISP_FRAUDE_00001, 190.10.99.99) y cantidades conocidas (50 cuentas, 20 ciclos), cada consulta de detección puede evaluarse preguntando si recupera exactamente el patrón sembrado, un subconjunto de él, o resultados contaminados con ruido.

Adicionalmente se contrastaron los resultados de cada requisito contra los de otro, para determinar si distintas técnicas convergen sobre las mismas cuentas; estas verificaciones cruzadas produjeron los hallazgos más informativos del trabajo (Secciones VII y VIII).

B. Tiempos de Ejecución

La Tabla IX reporta el tiempo de ejecución observado para cada consulta principal, medido en Memgraph Lab sobre la instancia descrita en la Sección III-C.

TABLA IX
Tiempos de Ejecución por Consulta

Consulta	Tiempo
Escala del grafo (nodos, relaciones, total de ciclos)	9.14 s
Umbral estadístico 3σ: diagnóstico	34 ms
Umbral estadístico 3σ: filtro aplicado	16 ms
IP compartida (criterio final)	80 ms
Verificación cruzada dispositivo/IP	19 ms
Ciclo y cadena lineal (LIMIT 10, sin ORDER BY)	18 ms*
Listado de procedimientos MAGE	2 ms
Ciclos con filtro de monto > $5,000	8.92 s
PageRank, top 10	34 ms
Verificación cruzada de PageRank	70 ms
Grado simple y su verificación cruzada	464 ms
Louvain, top de comunidades	389 ms
Verificación cruzada de Louvain	403 ms
Camino mínimo (BFS)	14 ms
Riesgo combinado	5 ms

Con LIMIT 10 y sin ORDER BY, Memgraph se detiene al encontrar los primeros diez resultados, de ahí el tiempo de 18 ms. Las consultas que deben recorrer todos los ciclos del grafo y, en su caso, ordenarlos —el conteo total, el filtro por monto o la versión con ORDER BY— tardan entre 9 y 16 s, como se detalla en la Sección VI-A.

C. Reproducibilidad

Todas las pruebas son reproducibles siguiendo el procedimiento documentado en el repositorio: levantar el contenedor de Memgraph con MAGE, ejecutar el script generador de datos (que produce siempre el mismo conjunto por la semilla fija), y ejecutar el archivo de consultas en el orden de los requisitos. Las quince mediciones de la Tabla IX se obtuvieron en una misma sesión y sobre el mismo equipo, el descrito en la Sección III-C, de modo que los tiempos son comparables entre sí; cada cifra proviene de una única ejecución y no de la mediana de varias repeticiones. Al no tratarse de un entorno de laboratorio controlado, los tiempos absolutos deben interpretarse como órdenes de magnitud y no como valores de referencia.

D. Resumen de Resultados

La Tabla X sintetiza el resultado de validación de cada requisito frente al patrón de fraude sembrado.

TABLA X
SÍNTESIS DE VALIDACIÓN POR REQUISITO

Requisito	Resultado	¿Aísla patrón?
Esquema del grafo	5 tipos de nodo y 5 de relación	N/A
Dispositivo compartido	1 resultado (50 clientes)	Sí
IP compartida	1 resultado (50 clientes)	Sí
Ciclo de transferencias	2 de 20 sembrados confirmados	Sí (parcial)
Centralidad	Valores uniformes	No
Comunidades	Dispersión en 19	No
Camino mínimo	5 saltos, 4 interm.	N/A
Riesgo combinado	Prioriza dentro del grupo	N/A (usa el ID conocido)

XIII. DISCUSIÓN Y LIMITACIONES

Los resultados obtenidos muestran patrones consistentes que vale la pena discutir explícitamente. Primero, las técnicas de centralidad (PageRank, grado simple) y detección de comunidades (Louvain) no aíslan el patrón de fraude sembrado; la identidad compartida (Sección V) sí lo aísla con claridad, y el patrón estructural sobre transferencias con montos altos (Sección VI) lo hace de forma parcial, recuperando 2 de los 20 ciclos sembrados. Esta divergencia no es un defecto: es evidencia de que cada técnica capta un ángulo distinto del fraude, y que la detección efectiva requiere combinar múltiples enfoques.

Segundo, la comparación de criterios para IP compartida (Sección V-B) ilustra que el filtrado por conteo simple es insuficiente cuando la relación subyacente tiene fan-out (una IP puede asociarse a varios dispositivos, y viceversa): aislar el patrón sembrado requirió una condición adicional sobre la estructura de esa relación, no solo sobre el volumen. El origen de ese fan-out es una decisión de modelado que conviene señalar como limitación propia: CONECTA_DESDE une Dispositivo con IP y no Cuenta con IP, de modo que el grafo no conserva desde qué dirección se conectó cada cuenta en particular. Un modelo que registrara la IP de cada conexión a nivel de cuenta habría hecho innecesario el segundo criterio. Dicho esto, la necesidad de una condición adicional es generalizable a cualquier consulta de identidad compartida sobre relaciones de muchos a muchos.

Tercero, el conjunto de datos es sintético y de distribuciones uniformes, por lo que no reproduce un grafo financiero real, donde la actividad transaccional sigue distribuciones de cola pesada y existen comunidades naturales por geografía o tipo de comercio. Es razonable suponer que sobre datos reales las técnicas de centralidad y comunidades se comportarían de otro modo.

Cuarto, se identificaron alternativas nativas de MAGE que no se implementaron por tiempo pero quedan como trabajo futuro: cycles.get para detección de ciclos, degree_centrality.get, y betweenness_centrality.get. Esta última podría ser particularmente útil para detectar cuentas que actúan como puentes en cadenas de lavado, un ángulo no cubierto por PageRank ni por el grado simple.

XIV. CONCLUSIONES

FraudeLink demuestra que las bases de datos de grafos son un vehículo natural para el análisis de fraude financiero. Los siete requisitos técnicos fueron implementados con resultados verificados empíricamente, incluyendo los casos en que ciertas técnicas no logran aislar el patrón sembrado, con explicación de por qué desde la estructura misma del dataset.

La implementación se apoya en Memgraph con MAGE, sobre un dataset sintético reproducible que supera los mínimos del caso. Se identificaron y resolvieron problemas técnicos reales (error de memoria en consultas de camino variable, limitación de sintaxis de Memgraph respecto a size() con patrones), y se iteró sobre resultados inesperados hasta lograr señales limpias. El trabajo futuro incluye la exploración de algoritmos MAGE adicionales, particularmente centralidad de intermediación, y la validación sobre datasets reales una vez disponibles.

El código fuente, los scripts de generación de datos, el archivo completo de consultas y las instrucciones de reproducción se encuentran disponibles en el repositorio público del proyecto: https://github.com/Jossygg11/Investigacion_grupal_1.

REFERENCIAS

[1] Universidad de Costa Rica, Escuela de Estadística, "Curso XS0131 — Gestión de Bases de Datos y Análisis de Información, Investigación Grupal 1, Caso FraudeLink," San José, Costa Rica, II Semestre 2026.

[2] I. Robinson, J. Webber, and E. Eifrem, *Graph Databases*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2015.

[3] Memgraph, "Cypher query language," Memgraph Documentation. [Online]. Available: https://memgraph.com/docs/cypher-manual (accessed Sep. 25, 2026).

[4] Memgraph, "MAGE — Memgraph Advanced Graph Extensions." [Online]. Available: https://memgraph.com/docs/mage (accessed Sep. 25, 2026).

[5] Memgraph, "Transactions," Memgraph Documentation. [Online]. Available: https://memgraph.com/docs/fundamentals/transactions (accessed Sep. 25, 2026).

[6] Memgraph, "Frequently asked questions," Memgraph Documentation. [Online]. Available: https://memgraph.com/docs/help-center/faq (accessed Sep. 25, 2026).

[7] Faker, "Faker documentation." [Online]. Available: https://faker.readthedocs.io (accessed Sep. 25, 2026).

[8] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Stanford, CA, USA, Tech. Rep. 1999-66, 1999.

[9] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Statist. Mech., Theory Exp.*, vol. 2008, no. 10, Oct. 2008, doi: 10.1088/1742-5468/2008/10/P10008.